


# University of Asia Pacific

## Department of Computer Science and Engineering

## CSE 450: VLSI Lab

# LAB REPORT

---

**Experiment Number:** 02

**Experiment Title:** Design and Simulation of a Full Adder Using NAND-Based Structural VHDL in Xilinx ISE


## Submitted by:

**Name:** MD Rawnak Shalahen Fahim

**Student ID:** 22201063

**Section:** C1


## Submitted to:

**Md. Rasheduzzaman**
**Lecturer**
**Department of Computer Science and Engineering**

**Date of Submission:** 10/25/2026

# 1. Title of the Experiment

**Design and Simulation of a Full Adder Using NAND-Based Structural VHDL in Xilinx ISE**

# 2. Objectives

The main objectives of this experiment are:

1. To understand the design and operation of basic digital logic gates.
2. To implement a **NAND gate using VHDL**.
3. To construct other logic gates using NAND gate instances.
4. To implement an **XOR gate using NAND gates**.
5. To design a **Half Adder using XOR and AND gates**.
6. To design a **Full Adder using two Half Adders and an OR gate**.
7. To understand **structural modeling** in VHDL.
8. To simulate the designed circuits using **Xilinx ISE**.
9. To verify the simulated outputs with the corresponding truth tables.
10. To understand hierarchical digital circuit design, where complex circuits are constructed from simpler reusable components.

# 3. Software and Tools Used

The following tools were used for this experiment:

- **Xilinx ISE 14.7**
- **VHDL**
- **ISim Simulator**
- Computer system for design and simulation

# 4. Introduction

A Full Adder is a fundamental combinational digital circuit used to perform the addition of three binary inputs. It has three inputs:

- A – First binary input
- B – Second binary input
- Cin – Carry input

and two outputs:

- Sum – Result of binary addition
- Cout – Carry output

The Boolean expressions for a Full Adder are:

$$Sum = A \oplus B \oplus C\ in$$
$$Cout = AB + (A \oplus B)Cin$$

In this experiment, the Full Adder was designed using a **hierarchical structural approach**. First, a NAND gate was implemented. NAND gates were then used to construct other basic gates. A Half Adder was constructed using XOR and AND gates, and finally a Full Adder was constructed using two Half Adders and an OR gate.

Therefore, the overall design follows the hierarchy:

NAND Gate
↓
Basic Logic Gates
↓
Half Adder
↓
Full Adder

## 5. NAND Gate Design

The NAND gate is a universal logic gate. A two-input NAND gate produces the inverse of the AND operation.

The Boolean expression is:

$$Y = \overline{A \cdot B}$$

**Truth Table**

| A | B | Y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

The NAND gate was implemented using VHDL Dataflow modeling.

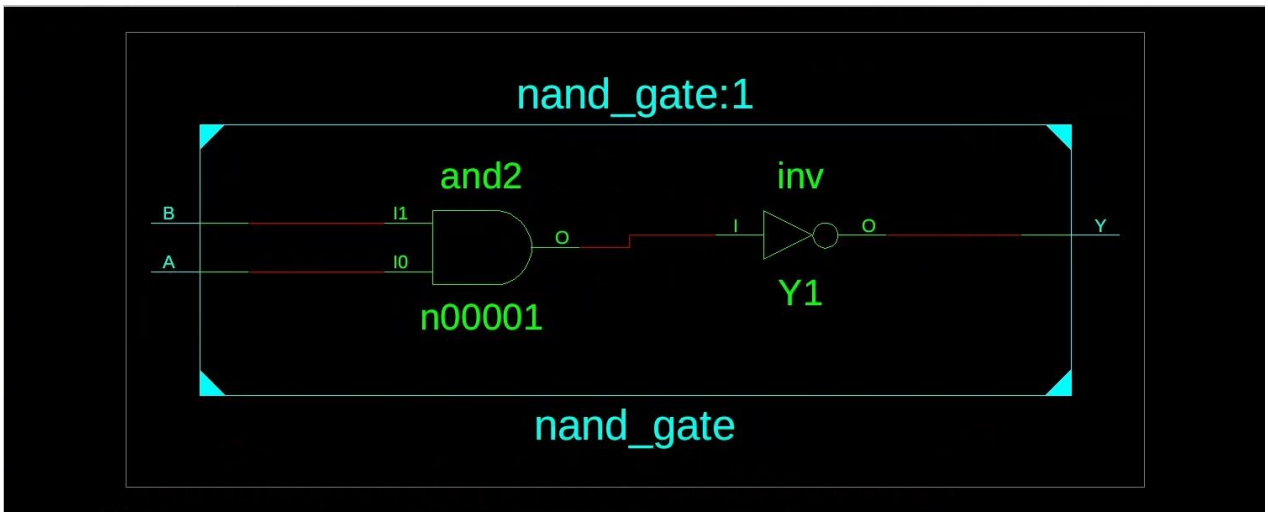



*Figure 1: RTL schematic of the NAND gate.*

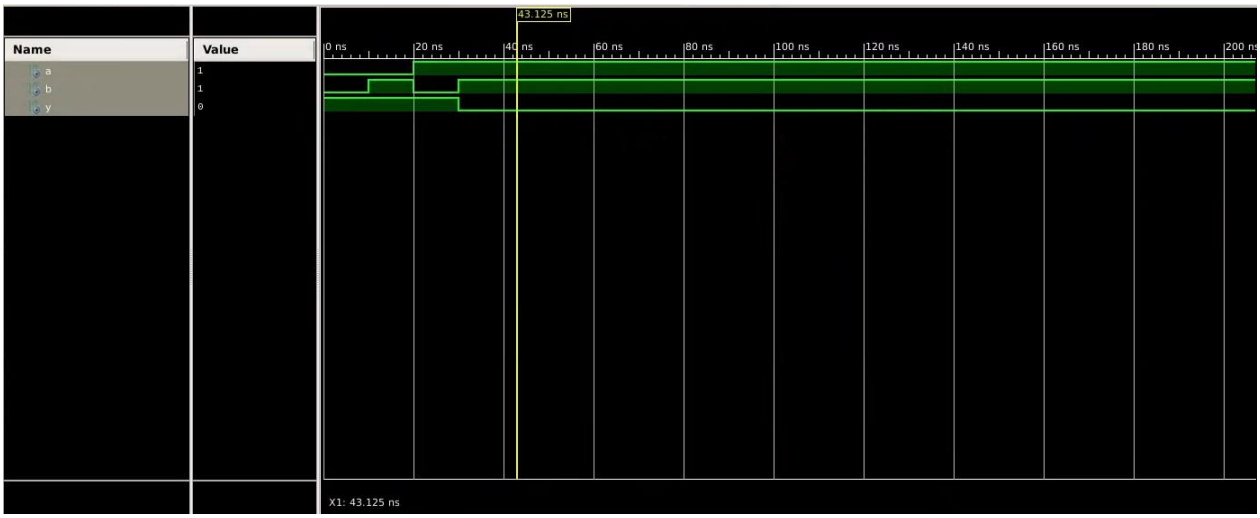



*Figure 2: Simulation waveform of the NAND gate.*

## 6. Construction of Basic Gates Using NAND Gates

Since NAND is a universal gate, other logic gates can be constructed using NAND gates.

The following gates were designed using NAND gate instances:

- AND
- OR
- NOR
- XOR

These gates were implemented using **structural VHDL modeling**.

## 6.1 AND Gate Using NAND Gates

An AND gate can be constructed using two NAND gates.

First:

$$N1 = ANANDB$$

Then:

$$Y = N1NANDN1$$

Therefore:

$$Y = A \cdot B$$

**Number of NAND gates**

**2 NAND gates**

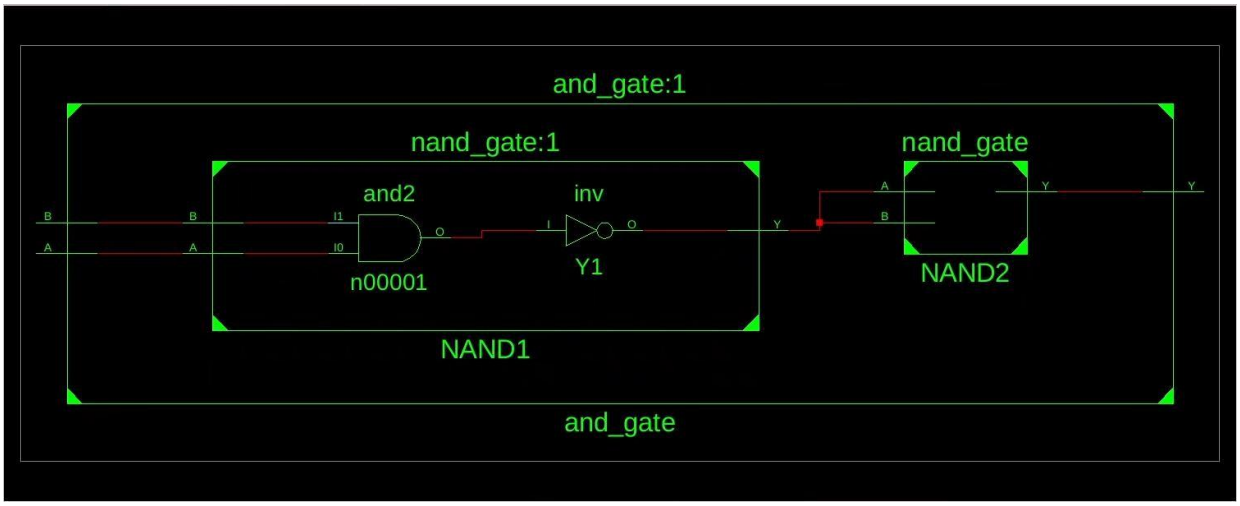



*Figure 3: RTL schematic of the NAND-based AND gate.*

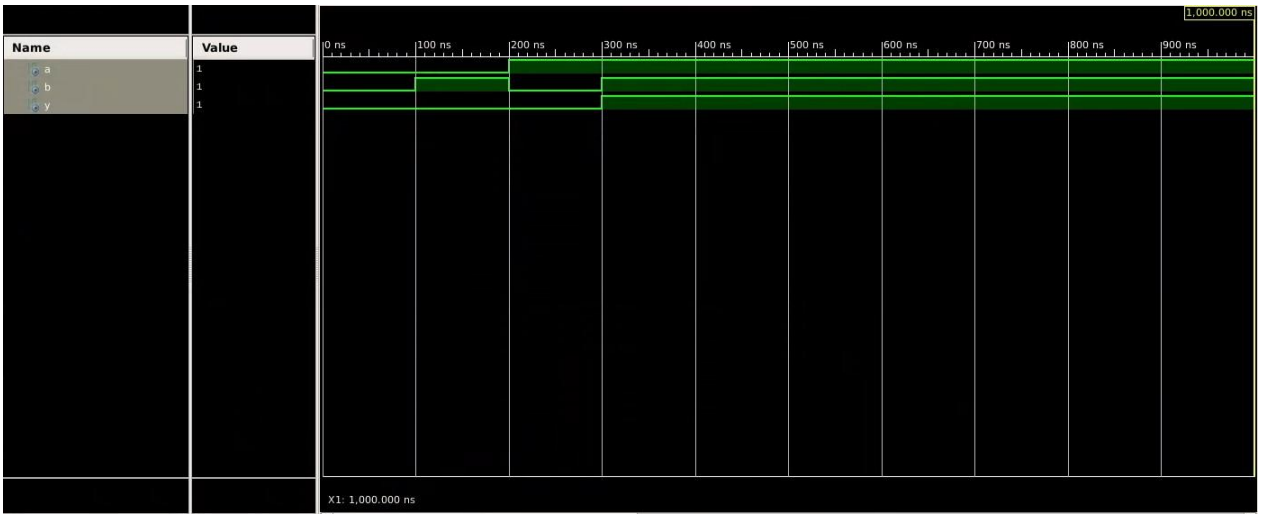



*Figure 4: Simulation waveform of the AND gate.*

## 6.2 OR Gate Using NAND Gates

Using De Morgan's theorem:

$$A + B = \bar{A}\cdot\bar{B}$$

Therefore:

N1 = A NAND A
N2 = B NAND B
Y = N1 NAND N2
Thus, an OR gate requires **3 NAND gates**.

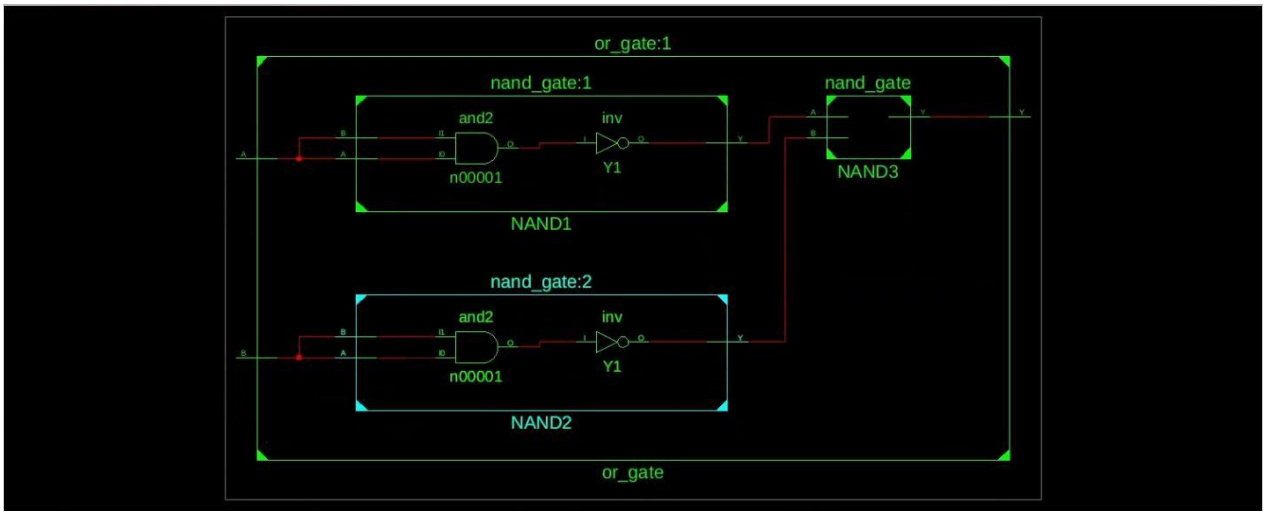



*Figure 5: RTL schematic of the NAND-based OR gate.*

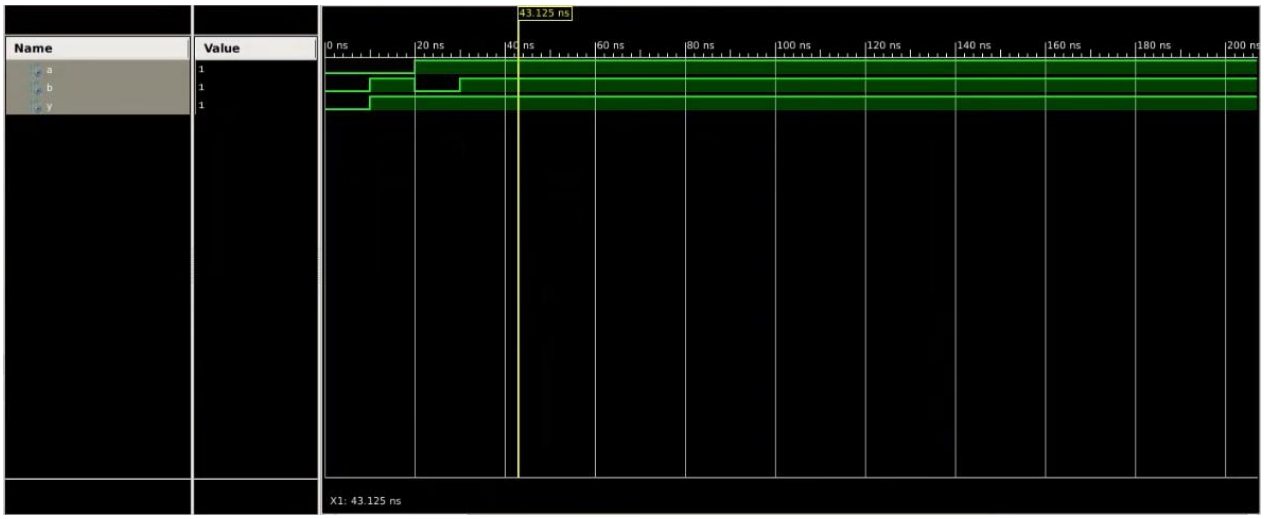



*Figure 6: Simulation waveform of the OR gate.*

## 6.3 NOR Gate Using NAND Gates

A NOR gate can be constructed by first generating an OR operation and then inverting the result.

The implementation uses four NAND gates:

N1 = A NAND A
N2 = B NAND B
N3 = N1 NAND N2
Y = N3 NAND N3
Therefore:

$$Y = A + B$$

**Number of NAND gates**

**4 NAND gates**

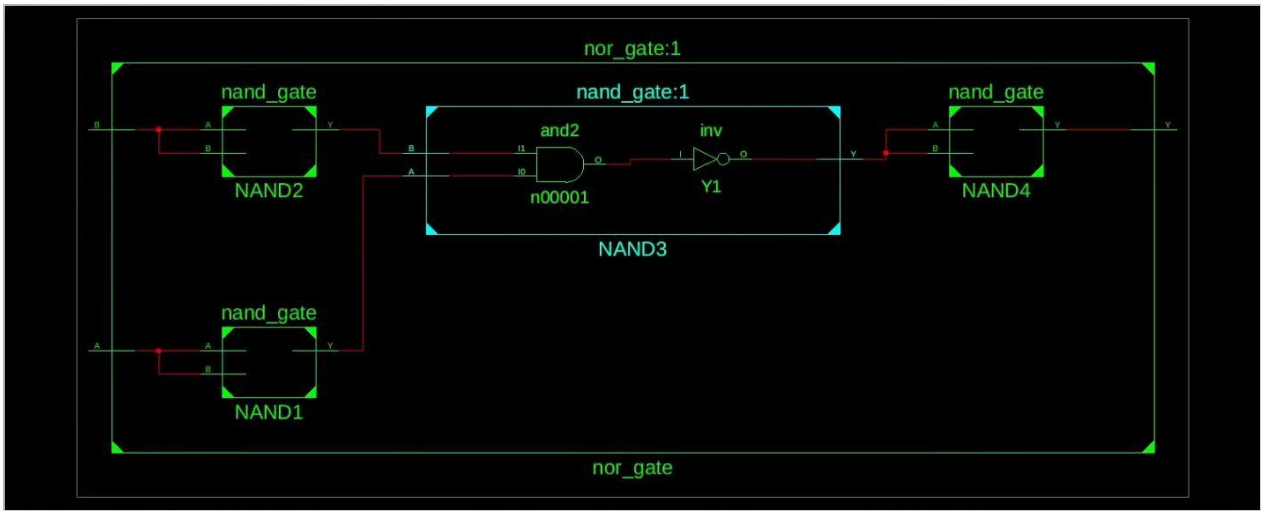



*Figure 7: RTL schematic of the NAND-based NOR gate.*

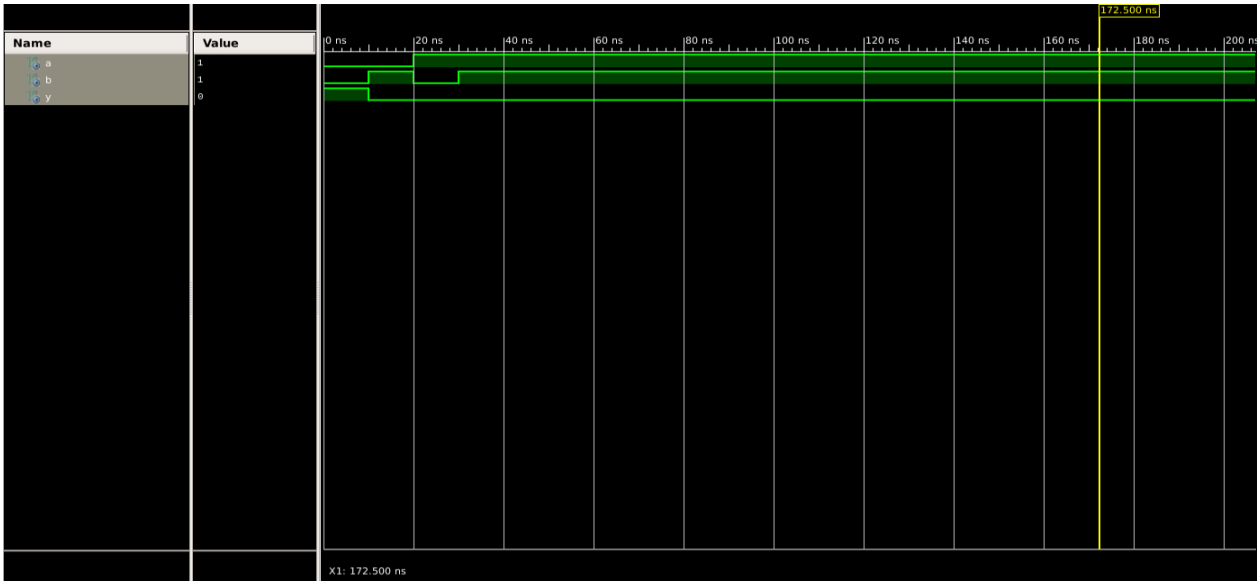



*Figure 8: Simulation waveform of the NOR gate.*

## 6.4 XOR Gate Using NAND Gates

The XOR gate produces logic 1 when the two inputs are different.

The NAND-only implementation uses four NAND gates:

N1 = A NAND B
N2 = A NAND N1
N3 = B NAND N1
Y = N2 NAND N3
Therefore:

$$Y = A \oplus B$$

**Truth Table**

| A | B | XOR |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

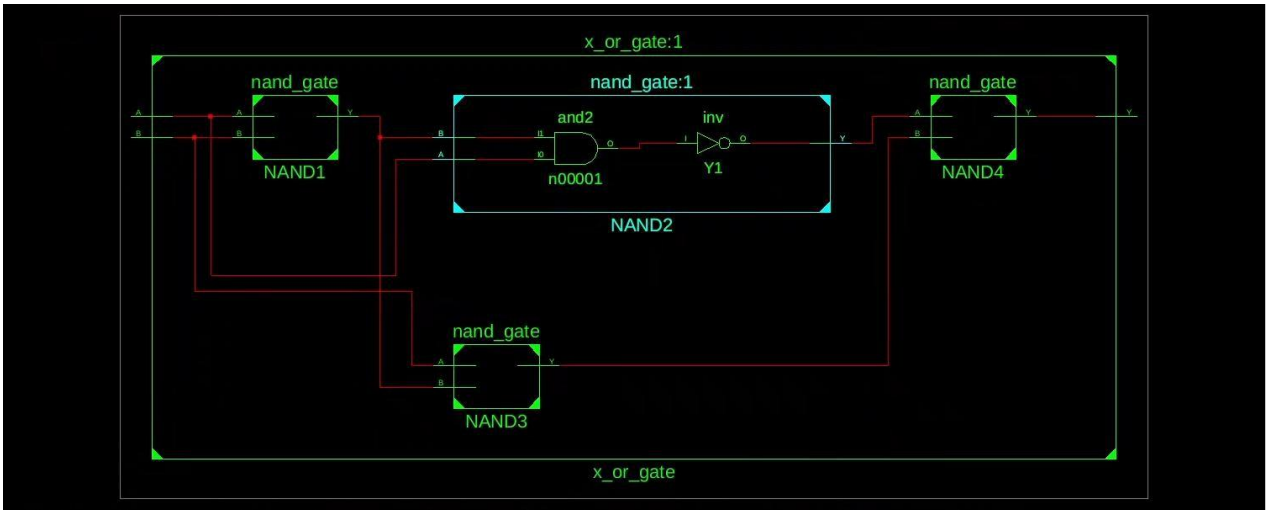



*Figure 9: RTL schematic of the NAND-based XOR gate.*

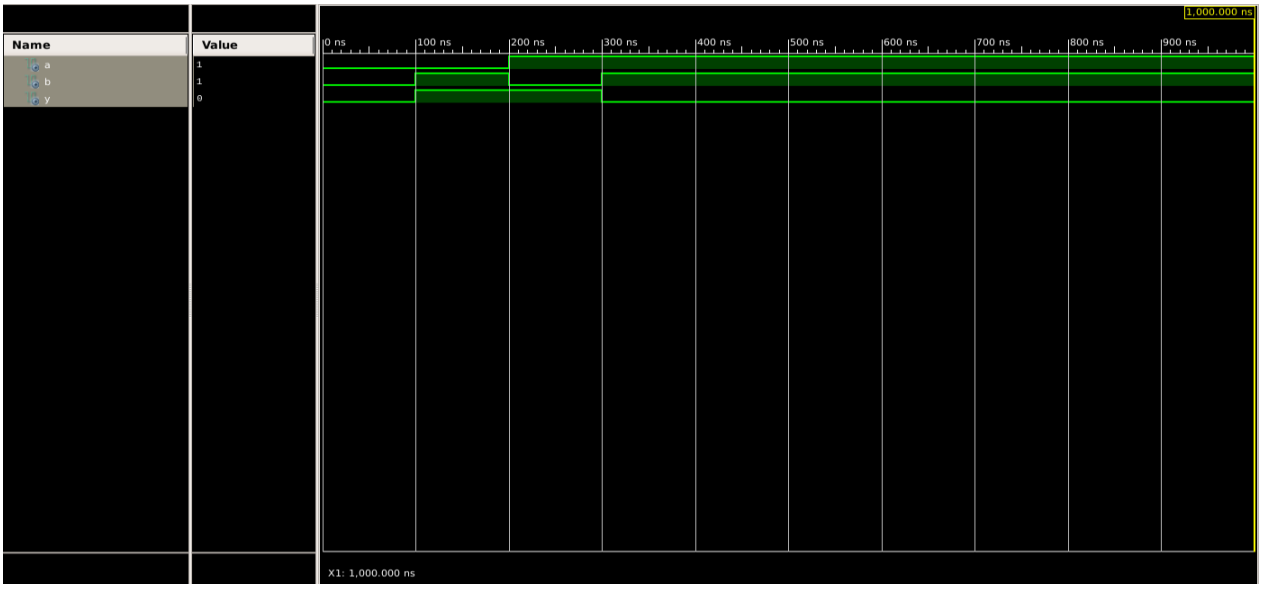



*Figure 10: Simulation waveform of the XOR gate.*

## 7. Half Adder Design

A Half Adder is a combinational circuit that adds two one-bit binary numbers.

It has:

**Inputs**

- A
- B

**Outputs**

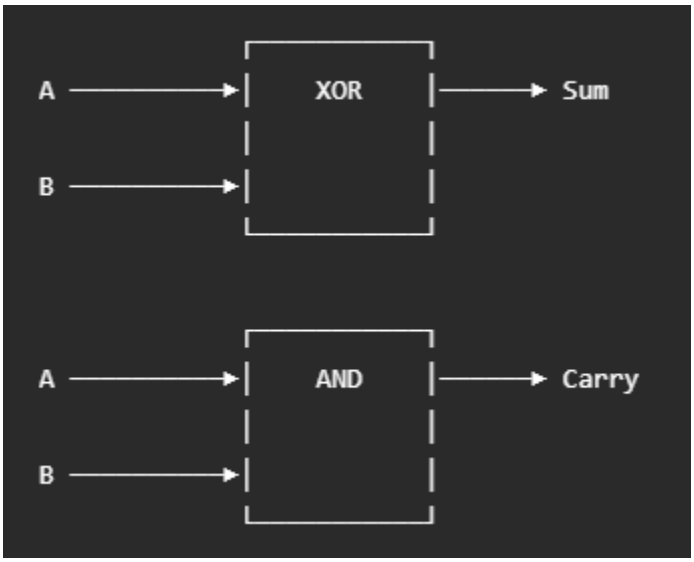
- Sum
- Carry

The Boolean expressions are:

$$Sum = A \oplus B$$
$$Carry = A \cdot B$$

The Half Adder was implemented structurally using the previously designed XOR and AND components.

**Block Diagram**




**Half Adder Truth Table**

| A | B | Sum | Carry |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |

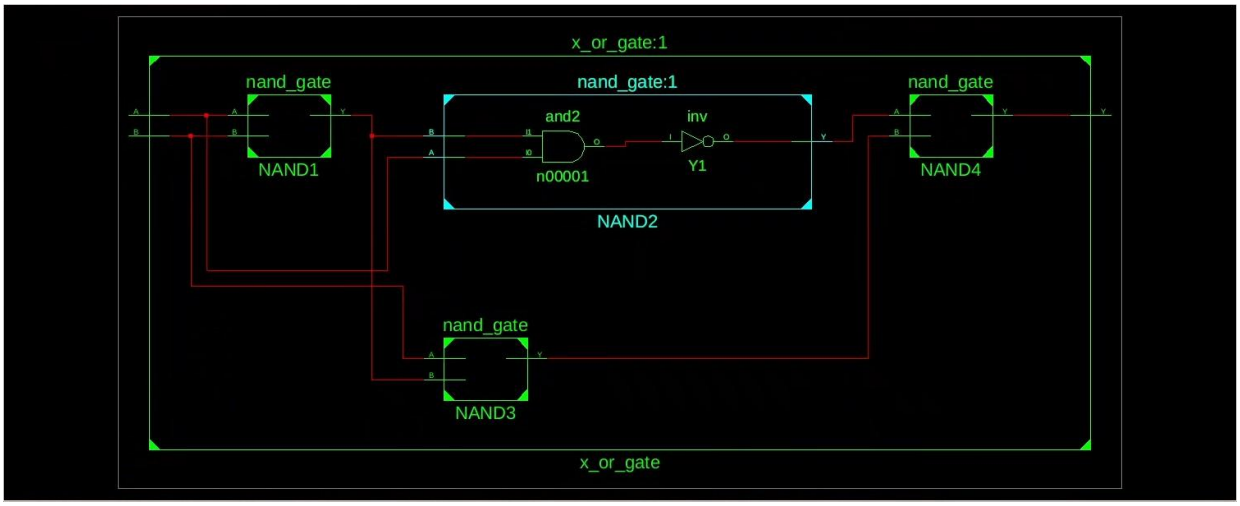



*Figure 11: RTL schematic of the Half Adder.*

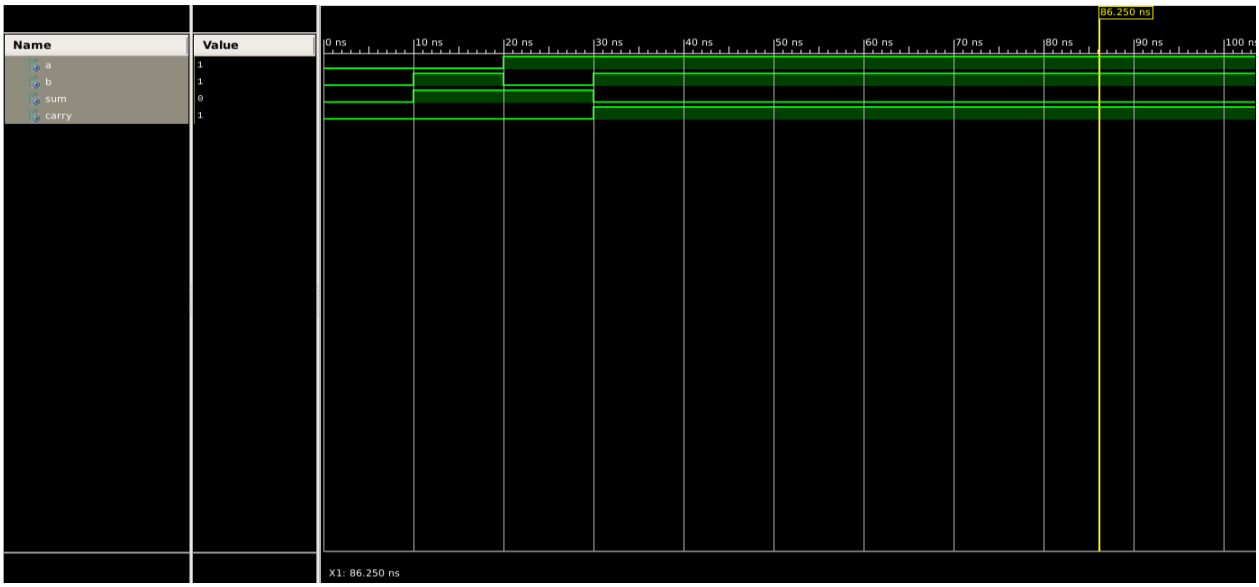



*Figure 12: Simulation waveform of the Half Adder.*

## 8. Full Adder Design

A Full Adder adds three one-bit binary values.

**Inputs**

- A
- B
- Cin

**Outputs**

- Sum
- Cout

A Full Adder can be constructed using:

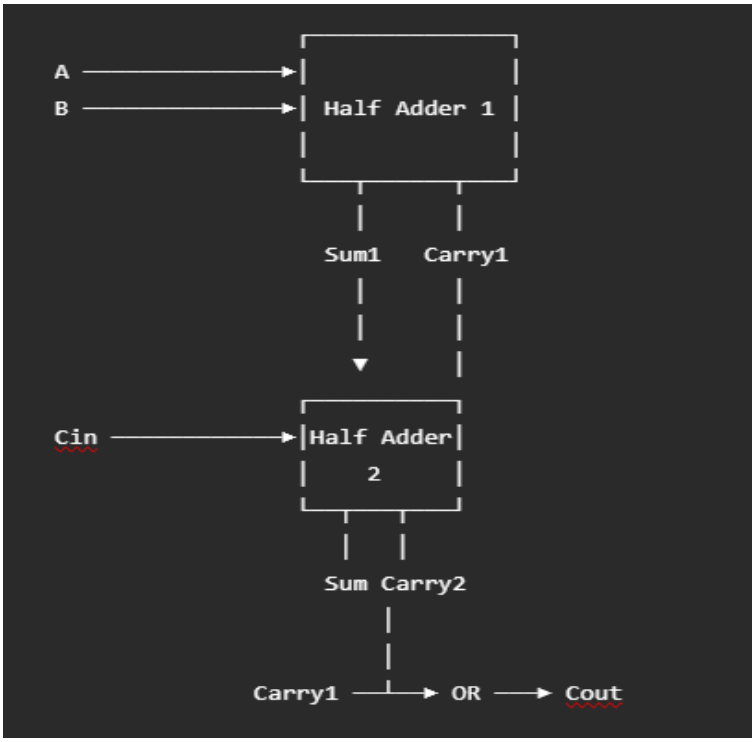
- Two Half Adders
- One OR gate

The first Half Adder adds A and B.

The second Half Adder adds the intermediate sum to Cin.

Finally, the two carry outputs are passed through an OR gate.

## 8.1 Full Adder Block Diagram




## 9. Full Adder Logic

The first Half Adder generates:

$$Sum1 = A \oplus B$$
$$Carry1 = A \cdot B$$

The second Half Adder generates:

$$Sum = Sum1 \oplus C\ in$$

$$Carry2 = Sum1 \cdot Cin$$

Finally:

$$Cout = Carry1 + Carry2$$

Therefore:

$$Sum = A \oplus B \oplus C\ in$$

and:

$$Cout = AB + (A \oplus B)Cin$$

## 10. Full Adder Truth Table

| A | B | Cin | Sum | Cout |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |

The simulation results should match this truth table for all eight possible input combinations.

## 11. RTL Schematic of the Full Adder

The final RTL schematic should show the hierarchical structure of the Full Adder.

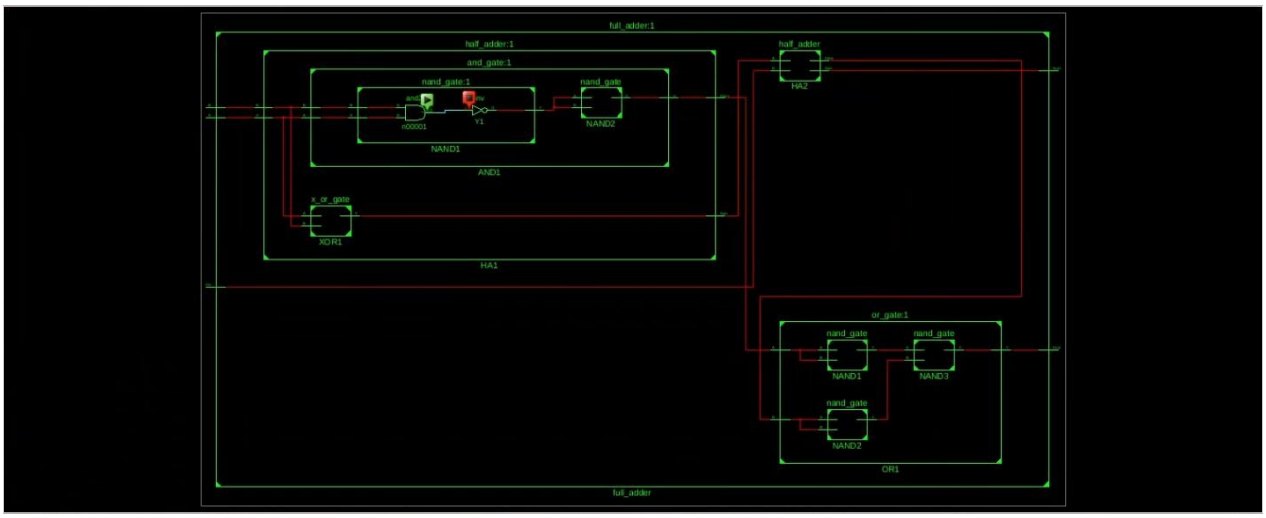



*Figure 13: RTL schematic of the complete Full Adder.*

This is one of the **most important screenshots in the report**, so make sure the RTL schematic is clearly visible.

## 12. Simulation

The complete Full Adder was simulated using the ISim simulator available in Xilinx ISE.

The testbench applies all eight possible combinations of the three inputs:

000 -> 0
001 -> 1
010 -> 2
011 -> 3
100 -> 4
101 -> 5
110 -> 6
111 -> 7
Each input combination is applied for a fixed time interval, allowing the corresponding Sum and Cout outputs to be observed.

**Full Adder Testbench**

The testbench verifies:

A = 0, B = 0, Cin = 0
A = 0, B = 0, Cin = 1
A = 0, B = 1, Cin = 0
A = 0, B = 1, Cin = 1
A = 1, B = 0, Cin = 0
A = 1, B = 0, Cin = 1
A = 1, B = 1, Cin = 0
A = 1, B = 1, Cin = 1

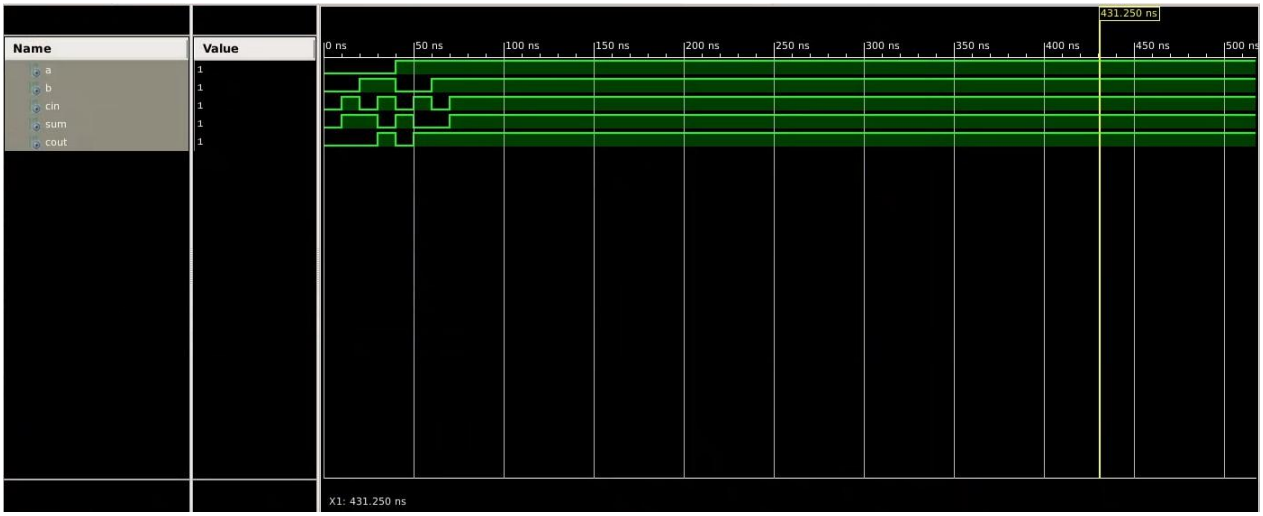



*Figure 14: Simulation waveform showing all eight input combinations and corresponding Sum and Cout outputs.*

For the waveform screenshot, make sure the following signals are visible:

A
B
Cin
Sum
Cout
It is preferable to zoom the waveform so that all eight input combinations can be easily identified.

## 13. Simulation Result Analysis

The simulation verifies the functionality of the designed Full Adder.

For example:

**Case 1**
A = 0
B = 0
Cin = 0

Sum = 0
Cout = 0
**Case 2**
A = 0
B = 1
Cin = 1

Sum = 0
Cout = 1
This represents:

$$0 + 1 + 1 = 10_2$$

Therefore:
Sum = 0
Cout = 1
**Case 3**
A = 1
B = 1
Cin = 1

Sum = 1
Cout = 1
This represents:

$$1 + 1 + 1 = 11_2$$

Therefore:
Sum = 1
Cout = 1
The simulation output agrees with the theoretical Full Adder truth table.

## 14. GitHub Repository

All VHDL source codes, testbenches, and related project files should be uploaded to a GitHub repository.

**GitHub Repository Link**

**https://github.com/rsf-rawnak/CSE450-VLSI-Labs**

## 15. Conclusion

In this experiment, a complete **Full Adder circuit was designed and simulated using VHDL and Xilinx ISE**. The design followed a hierarchical structural approach.

Initially, a NAND gate was implemented. Since NAND is a universal logic gate, it was then used to construct other logic gates such as AND, OR, NOR, and XOR. These gates were used to construct a Half Adder, which was subsequently used to construct the final Full Adder.

The Full Adder was implemented using two Half Adders and one OR gate. The resulting circuit accepts three binary inputs (A, B, and Cin) and produces the Sum and Cout outputs.

Simulation was performed for all eight possible combinations of the three input signals. The obtained simulation results matched the theoretical Full Adder truth table. Therefore, the design was successfully verified.

The experiment also demonstrated how **structural VHDL modeling, component instantiation, hierarchical design, and simulation** can be used to develop complex digital circuits from simpler reusable building blocks.